

LLMs & VLMs course

Pretraining on unlabeled data

Juan Irving Vasquez

October 21, 2026

Instituto Politécnico Nacional (IPN)

Contents

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

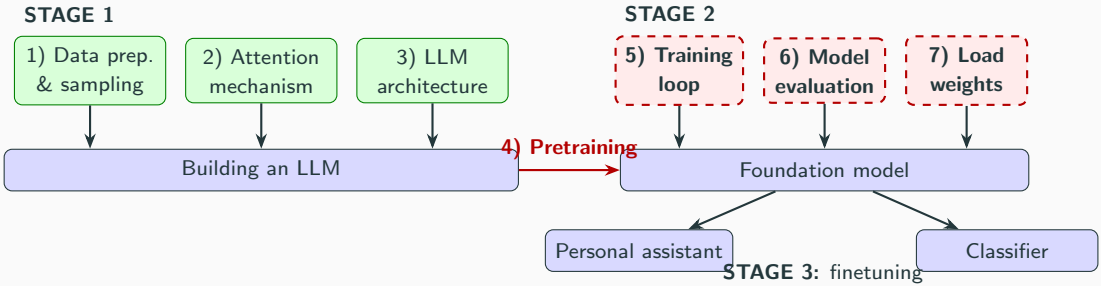
Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

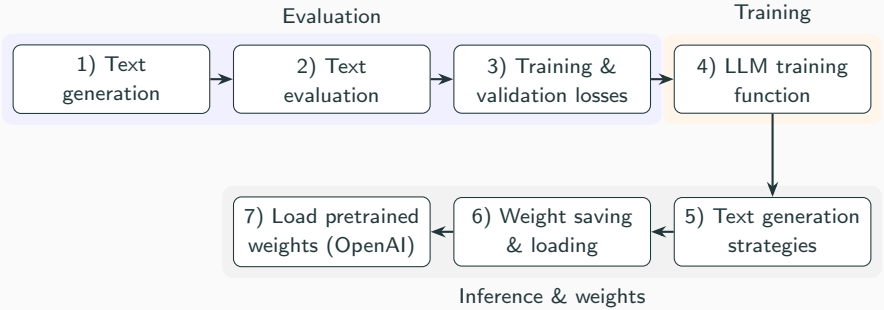
Summary and exercises

Where are we?



Last session: the **GPT architecture** (random weights ⇒ gibberish). Today: **train it**, evaluate it, and load **OpenAI's pretrained weights**.

Plan for this session



Notation

Symbol	Meaning
$\mathcal{V}$, $V = \lvert\mathcal{V}\rvert$	vocabulary, $V = 50{,}257$
$x_1, \ldots, x_T$	input token IDs, $x_t \in \mathcal{V}$
$x_{<t}$	context $x_1, \ldots, x_{t-1}$
d	embedding dimension (768)
$\mathbf{h}_t \in \mathbb{R}^d$	final hidden state at position t
$\mathbf{z}_t \in \mathbb{R}^V$	logits at position t
θ	all trainable parameters
$p_\theta(v \mid x_{<t})$	model probability of token v
$\mathbf{e}_y \in \{0,1\}^V$	one-hot vector of token y

- Vectors in **bold**, matrices in capitals (W, E).
- PyTorch tensors add a **batch** dimension B: `[B, T, d]`, `[B, T, V]`.
- log is the **natural** logarithm (as in `torch.log`).

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

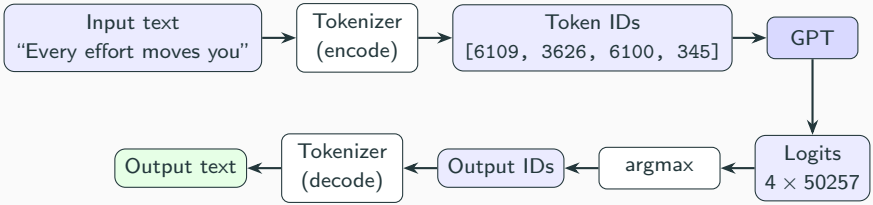
Summary and exercises

Setting up the model

Setting	Value
Vocabulary size	50,257
Context length	**256** (was 1024)
Embedding dim.	768
Attention heads	12
Transformer blocks	12
Dropout rate	0.1
Q, K, V bias	False

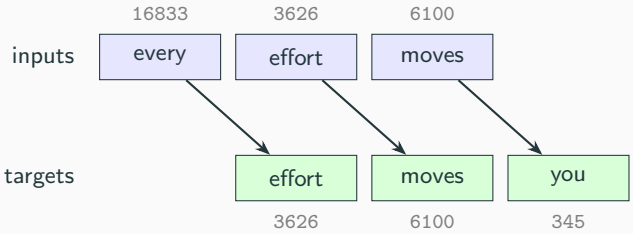
- Same `GPTModel` (GPT-2 small).
- Only change: context length $1024 \rightarrow 256$.
- Less compute and memory $\Rightarrow$ training fits on a **standard laptop**.
- At the end of the chapter we switch back to 1024 to load OpenAI's weights.

Text → token IDs → logits → text



Two helper functions make this cycle easy to reuse throughout the chapter: `text_to_token_ids` and `token_ids_to_text`.

Inputs and targets



- Batch of two examples: "every effort moves" → "effort moves **you**" and "I really like" → "really like **chocolate**".
- Targets = inputs **shifted one position** (data loader from Chapter 2).
- At **every** position the model must predict the **next token**.

Language modeling as a probability problem

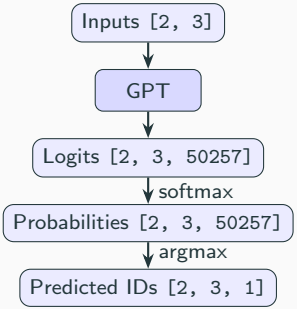
By the **chain rule of probability**, any sequence can be factorized as

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P\big(x_t \mid x_{<t}\big)$$

$$\begin{aligned} P(\text{every effort moves you}) = {} & P(\text{every}) \cdot P(\text{effort} \mid \text{every}) \\ & \cdot P(\text{moves} \mid \text{every effort}) \cdot P(\text{you} \mid \text{every effort moves}) \end{aligned}$$

- A GPT **approximates each conditional**: $p_\theta(x_t \mid x_{<t}) \approx P(x_t \mid x_{<t})$ (**autoregressive** model).
- Thanks to the **causal mask**, one forward pass over T tokens returns all T conditionals at once.
- This is why the targets are the inputs **shifted by one**: $y_t = x_{t+1}$.

From logits to predicted tokens



- Shape `[batch, tokens, vocab]`.
- One distribution over **50,257 tokens** per position.

Targets:	`effort moves you`
Outputs:	`Armed heNetflix`

Goal: **maximize the probability** of the correct target token at every position.

From hidden states to probabilities

Output layer (linear projection to the vocabulary):

$$\mathbf{z}_t = W_{\text{out}}\,\mathbf{h}_t, \qquad W_{\text{out}} \in \mathbb{R}^{V \times d}$$

Softmax turns logits into a distribution:

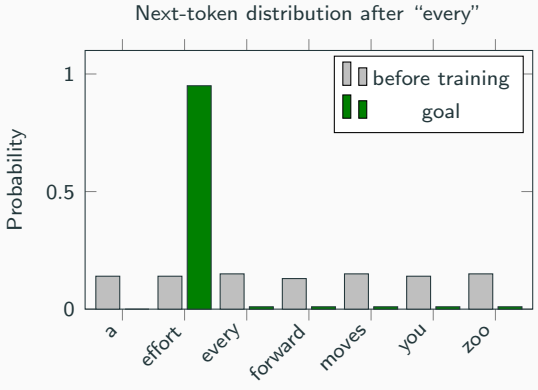
$$p_\theta(v \mid x_{\le t}) = \text{softmax}(\mathbf{z}_t)_v = \frac{e^{z_{t,v}}}{\sum_{j=1}^{V} e^{z_{t,j}}}$$

Greedy decoding:

$$\hat{y}_t = \arg\max_{v \in \mathcal{V}} \; p_\theta(v \mid x_{\le t}) = \arg\max_{v} \; z_{t,v}$$

- $p_v > 0$ and $\sum_v p_v = 1$.
- Softmax is **monotonic**: argmax of probabilities = argmax of logits.
- **Shift invariance**: $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$. Implementations subtract $\max_j z_j$ to avoid overflow of e^z.
- Batched: $H \in \mathbb{R}^{B \times T \times d} \to Z = HW_{\text{out}}^\top \in \mathbb{R}^{B \times T \times V}$.

Probabilities of the target tokens



Toy 7-token vocabulary: random start $\approx 1/7$.

- Probability of each target token (untrained):
  - Text 1: $7.5\times10^{-5},\ 3.1\times10^{-5},\ 1.2\times10^{-5}$
  - Text 2: $4.0\times10^{-5},\ 1.7\times10^{-5},\ 4.8\times10^{-6}$
- Close to $1/50{,}257 \approx 0.00002$.
- We want them to move **towards 1**, by updating weights with **backpropagation**.
- Backpropagation needs a **loss function**.

Maximum likelihood: why logarithms?

Training = find the parameters that make the observed targets **most probable**:

$$\theta^* = \arg\max_\theta \prod_{i=1}^{N} p_\theta(y_i \mid x_{\le i}) = \arg\min_\theta \; \mathcal{L}(\theta)$$

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid x_{\le i}) \qquad \text{(average negative log-likelihood = cross entropy)}$$

Why the log?

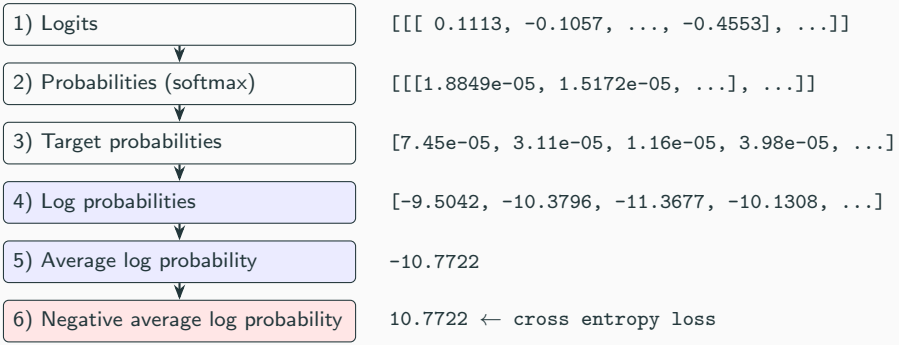
- log is strictly increasing $\Rightarrow$ the **same maximizer**.
- Turns a **product** into a **sum** (easier gradients).
- Dividing by N and negating: an **average** to **minimize**.

Numerical example (our 6 targets):

$$\prod_i p_i = e^{-64.63} \approx 8.5 \times 10^{-29}$$

With thousands of tokens the product **underflows** float32 ($\approx 10^{-38}$); the sum of logs does not.

Computing the loss step by step



Working with **log**-probabilities is numerically more convenient for optimization. By convention, we **minimize** the negative value instead of maximizing it.

Cross entropy loss

For N token positions with targets $y_1, \ldots, y_N$ and predicted distributions $p_\theta(\cdot)$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta\big(y_i \mid x_1, \ldots, x_i\big)$$

- Compares the **true** (one-hot) and the **predicted** distributions.
- Equivalent to the **negative average log probability** of the targets.
- $\mathcal{L} = 0 \Leftrightarrow$ the model assigns probability 1 to every target token.
- Untrained model: $\mathcal{L} \approx \log(50{,}257) \approx 10.8$ (uniform guess).
- In practice: flatten batch and positions ($2 \times 3 = 6$ examples), apply cross entropy to raw logits.

Cross entropy, entropy and KL divergence

For a true distribution q and a predicted distribution p over $\mathcal{V}$:

$$H(q,p) = -\sum_{v\in\mathcal{V}} q(v)\log p(v) \;=\; \underbrace{H(q)}_{\text{entropy of } q} \;+\; \underbrace{D_{\mathrm{KL}}(q\,\|\,p)}_{\geq 0}$$

One-hot target $q = \mathbf{e}_y$:

- $H(q,p) = -\log p(y)$: only the target term survives.
- $H(q) = 0 \Rightarrow$ minimizing cross entropy = minimizing $D_{\mathrm{KL}}(q\,\|\,p)$.
- Minimum 0 when $p(y) = 1$.

Uniform baseline $p(v) = 1/V$:

$$\mathcal{L} = -\log \tfrac{1}{V} = \log V = \log 50{,}257 \approx 10.82$$

Our untrained model: $\mathcal{L} = 10.77 \Rightarrow$ it is essentially **guessing uniformly**.

Gradient of the cross entropy

For one position with logits $\mathbf{z}$, $\mathbf{p} = \mathrm{softmax}(\mathbf{z})$ and target y:

$$\mathcal{L} = -\log \frac{e^{z_y}}{\sum_j e^{z_j}} = -z_y + \log \sum_j e^{z_j} \quad \Longrightarrow \quad \frac{\partial \mathcal{L}}{\partial z_v} = p_v - \mathbb{1}[v = y], \quad \nabla_{\mathbf{z}} \mathcal{L} = \mathbf{p} - \mathbf{e}_y$$

Interpretation (gradient descent moves *against* the gradient):

- Target logit z_y is pushed **up** by $1 - p_y$.
- Every other logit z_v is pushed **down** by p_v.
- The gradient vanishes only when $p_y = 1$.

Backpropagation (chain rule) into the output layer, $\mathbf{z} = W_{\text{out}}\mathbf{h}$:

$$\frac{\partial \mathcal{L}}{\partial W_{\text{out}}} = (\mathbf{p} - \mathbf{e}_y)\,\mathbf{h}^\top \in \mathbb{R}^{V \times d}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = W_{\text{out}}^\top (\mathbf{p} - \mathbf{e}_y) \in \mathbb{R}^d$$

and so on, layer by layer, down to the embeddings.

Perplexity

$$\mathrm{PPL} = \exp\left(\mathcal{L}_{\mathrm{CE}}\right) = \exp\Big(-\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(y_i \mid x_{\le i})\Big) = \Big(\prod_{i=1}^{N}\frac{1}{p_\theta(y_i \mid x_{\le i})}\Big)^{1/N}$$

uniform model: $\mathrm{PPL} = V = 50{,}257$

- The **geometric mean** of $1/p$ over the targets: a more **interpretable** version of the loss.
- It is the **effective vocabulary size** the model is uncertain about at each step.
- Our untrained model is "choosing" among $\approx$ **47,678 tokens** (almost the full vocabulary).
- Lower is better; a perfect model has $\mathrm{PPL} = 1$.

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

The dataset: The Verdict

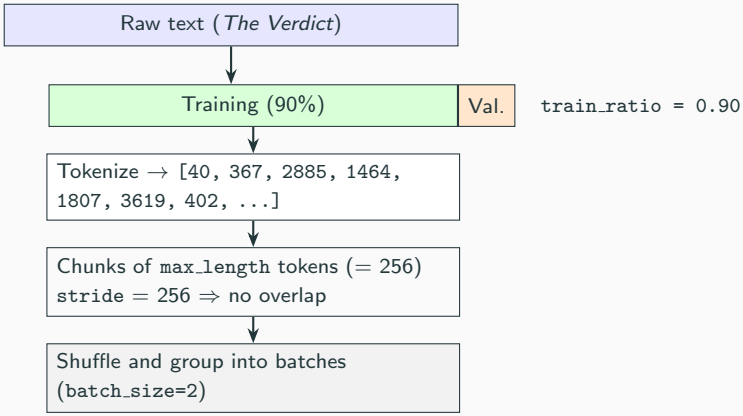
- Short story by Edith Wharton (public domain), used in Chapter 2.
- **20,479** characters, **5,145** tokens.
- Tiny on purpose: training takes **minutes** on a laptop.
- Bigger option: 60,000+ books from Project Gutenberg (Appendix D).

The cost of pretraining
Llama 2 (7B parameters):

- 184,320 A100 GPU hours
- 2 trillion tokens
- 8×A100 server ≈ \$30/hour

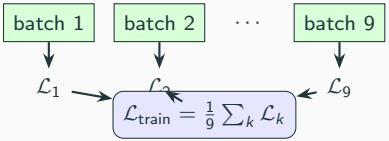
$$\frac{184{,}320}{8} \times \$30 \approx \mathbf{\$690{,}000}$$

Preparing the data loaders



In practice: **variable-length** inputs help generalization, and real LLMs use batch sizes of 1,024 or more.

Loss over a whole dataset



- Loss of one batch: cross entropy over all its token positions.
- Loss of a data loader: **average** over its batches (optionally only the first n batches, for speed).
- Same procedure for training and validation sets.

Before training	Loss
Training set	10.988
Validation set	10.981

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

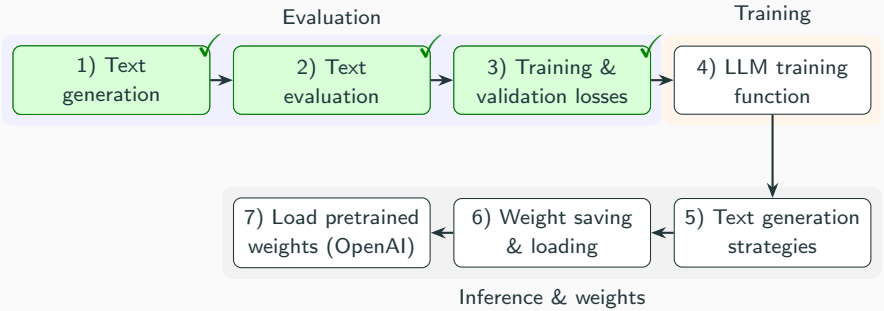
Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Progress so far



A typical PyTorch training loop

1) Iterate over training **epochs**

   2) Iterate over **batches** in the training set

      3) Reset gradients from the previous iteration — `optimizer.zero_grad()`

      4) Calculate the **loss** on the current batch — `calc_loss_batch(...)`

      5) **Backward pass** to compute loss gradients — `loss.backward()`

      6) **Update** weights using the gradients — `optimizer.step()`

   7) Print training and validation losses — `evaluate_model(...)`

8) Generate sample text for visual inspection — `generate_and_print_sample(...)`

The AdamW optimizer

- **Adam**: adaptive learning rate per parameter, using running averages of the gradient (momentum) and of its square.
- **AdamW**: variant with **decoupled weight decay**.
  - Weight decay penalizes large weights $\Rightarrow$ less model complexity, less overfitting.
  - In AdamW the decay is applied directly to the weights, not mixed into the adaptive gradient.
- Better regularization and generalization $\Rightarrow$ the **standard choice** for training LLMs.

Plain (mini-batch) gradient descent: $\theta_t = \theta_{t-1} - \eta \nabla_\theta \mathcal{L}_{\text{batch}}(\theta_{t-1})$

The AdamW update rule

At step t, with gradient $\mathbf{g}_t = \nabla_\theta \mathcal{L}_{\text{batch}}(\theta_{t-1})$:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t$$
$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2$$
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$$
$$\theta_t = \theta_{t-1} - \eta\Big(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\,\theta_{t-1}\Big)$$

All operations are **element-wise**. Here:
$\eta = 4\times10^{-4}$, $\lambda = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

- $\mathbf{m}_t$: exponential moving average of gradients (**momentum**).
- $\mathbf{v}_t$: moving average of squared gradients $\Rightarrow$ **per-parameter** step size $\eta/\sqrt{\hat{\mathbf{v}}_t}$.
- Bias correction: $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$ would bias early estimates towards 0.
- **Adam + L2**: adds $\lambda\theta$ to $\mathbf{g}_t$, so the decay is also divided by $\sqrt{\hat{\mathbf{v}}_t}$.
- **AdamW**: the decay term $\lambda\theta$ is **decoupled**, the same for all weights.
- $\mathbf{m}_t, \mathbf{v}_t$ are the optimizer **state** we save in checkpoints.

Running the training

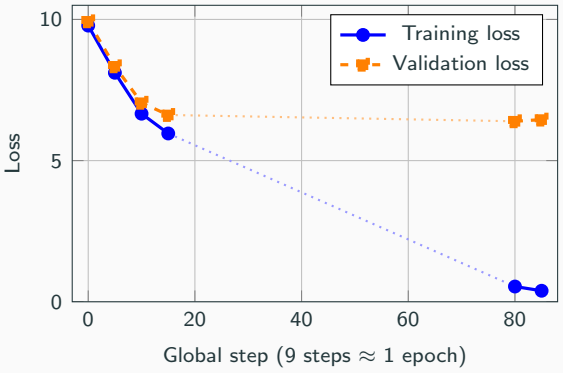
Hyperparameter	Value
Optimizer	AdamW
Learning rate	4×10^{-4}
Weight decay	0.1
Epochs	10
Batch size	2
Eval. every	5 steps

About **5 minutes** on a laptop.

```
Ep 1 (Step 000000):  Train loss 9.781, Val loss 9.933
Every effort moves you,,,,,,,,,,,,.
Ep 2 (Step 000015):  Train loss 5.961, Val loss 6.616
Every effort moves you, and, and, and, and, and, ...
[...]
Ep 9 (Step 000080):  Train loss 0.541, Val loss 6.393
Every effort moves you?" "Yes--quite insensible to the irony.  ...
Ep 10 (Step 000085):  Train loss 0.391, Val loss 6.452
Every effort moves you know," was one of the axioms he ...
```

From commas and repeated “and” to grammatically correct sentences.

Training vs. validation loss



Points printed in the training log (steps 20–75 omitted, dotted).

- Both losses drop sharply at first ⇒ the model is learning.
- After ∼2 epochs, training loss keeps falling but validation loss **stagnates** (≈ 6.4).
- Clear sign of **overfitting**: the model **memorizes** the text.
- "*quite insensible to the irony*" appears verbatim in *The Verdict*.

Why does it overfit?

- The dataset is **very, very small** (5,145 tokens) for a model with 124M+ parameters.
- We train for **10 epochs** over the same text.
- Real LLMs are trained on huge corpora, often for **about one epoch**.
- Training on 60,000 Gutenberg books does not show this overfitting.

More advanced training (Appendix D)
Learning-rate warmup, cosine annealing, and gradient clipping.

Next: **decoding strategies** to generate more **novel** text instead of memorized passages.

Overfitting, formally

Expected risk (what we care about) vs. **empirical risk** (what we minimize):

$$R(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[-\log p_\theta(y \mid x)\big]$$

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} -\log p_\theta(y_i \mid x_{\leq i})$$

The validation loss $\hat{R}_{\text{val}}$ is an **unbiased estimate** of $R(\theta)$ (data not used for training).

$$\text{gap} = \hat{R}_{\text{val}} - \hat{R}_{\text{train}} \approx 6.45 - 0.39 = 6.06$$

Parameters vs. data

- Model: $\approx$ 162M parameters.
- Training data: $9 \times 2 \times 256 = 4{,}608$ tokens.
- $> 35{,}000$ parameters **per training token** $\Rightarrow$ enough capacity to memorize.
- Compute-optimal rule of thumb (Chinchilla): ≈ 20 tokens per parameter $\Rightarrow$ $\sim$3B tokens for this model.

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Greedy decoding is deterministic

- `generate_text_simple` always takes the token with the **highest probability** (`argmax`) ⇒ **greedy decoding**.
- Same prompt ⇒ **always the same output**:

```
Every effort moves you know," was one of the axioms he laid down across the
                                    ...
```

- Two techniques to control randomness and diversity:
  1. **Temperature scaling**
  2. **Top-k sampling**
- Inference does not need a GPU for this model: `model.to("cpu")`, `model.eval()`.

A toy example: from argmax to sampling

Context: "*every effort moves you*" (9-token vocabulary)

	closer	every	effort	forward	inches	moves	pizza	toward	you
Logits	4.51	0.89	-1.90	**6.75**	1.63	-1.62	-1.89	6.28	1.79
Softmax	0.06	0.00	0.00	**0.57**	0.00	0.00	0.00	0.36	0.00
Sampled (1000×)	73	0	0	**582**	2	0	0	343	0

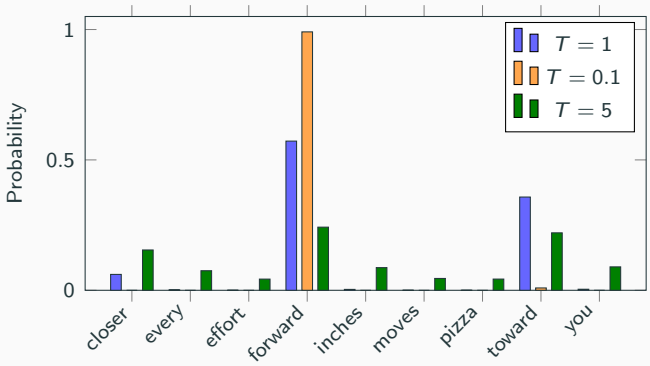
- **Argmax** (greedy): always "forward".
- **Multinomial sampling**: the next token is a random variable with a **categorical** distribution, $X \sim \mathrm{Categorical}(p_1, \ldots, p_V)$, $P(X = v) = p_v$
  $\Rightarrow$ "forward" most of the time, but also "toward", "closer", ...

Temperature scaling

Divide the logits by a temperature $T > 0$:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T = 1$: original distribution.
- $T < 1$: **sharper** $\rightarrow$ close to argmax.
- $T > 1$: **flatter** $\rightarrow$ more diverse, but more nonsense.



Temperature: the math

$$p_v(T) = \frac{e^{z_v/T}}{\sum_j e^{z_j/T}}, \qquad \frac{p_i(T)}{p_j(T)} = \exp\Big(\frac{z_i - z_j}{T}\Big)$$

Limits:

$$\lim_{T\to 0^+} \mathbf{p}(T) = \mathbf{e}_{\arg\max_v z_v} \qquad \text{(greedy)}$$

$$\lim_{T\to\infty} p_v(T) = 1/V \qquad \text{(uniform)}$$

Entropy $H(\mathbf{p}) = -\sum_v p_v \log p_v$ **increases with** T: from 0 (deterministic) to $\log V$ (maximal uncertainty).

Toy example: "forward" vs. "toward" ($z_i - z_j = 0.47$)

T	$p_{\text{forward}}/p_{\text{toward}}$	$H(\mathbf{p})$
0.1	$e^{4.7} \approx 110$	0.05
1	$e^{0.47} \approx 1.60$	0.91
5	$e^{0.094} \approx 1.10$	2.00
∞	1	$\log 9 = 2.20$

$T = 1$ is the distribution learned by **maximum likelihood**; $T \neq 1$ deliberately distorts it.

Top-k sampling

Keep only the k tokens with the largest logits; mask the rest with $-\infty$ before the softmax (the same trick as the **causal mask** in Chapter 3).

	closer	every	effort	forward	inches	moves	pizza	toward	you
Logits	4.51	0.89	-1.90	6.75	1.63	-1.62	-1.89	6.28	1.79
Top-3	4.51	$-\infty$	$-\infty$	6.75	$-\infty$	$-\infty$	$-\infty$	6.28	$-\infty$
Softmax	**0.06**	0	0	**0.58**	0	0	0	**0.36**	0

- Non-top-k tokens get probability **exactly 0** $\Rightarrow$ no more "*every effort moves you pizza*".
- The remaining probabilities sum to 1; combine with temperature + multinomial sampling.

Top-k: the math

Let $\mathcal{V}_k \subset \mathcal{V}$ be the k tokens with the largest logits. The top-k distribution is

$$\tilde{p}_v = \frac{e^{z_v/T}\,\mathbb{1}[v \in \mathcal{V}_k]}{\sum_{j\in\mathcal{V}_k} e^{z_j/T}} = \begin{cases} p_v \big/ \sum_{j\in\mathcal{V}_k} p_j, & v \in \mathcal{V}_k \\ 0, & \text{otherwise} \end{cases}$$

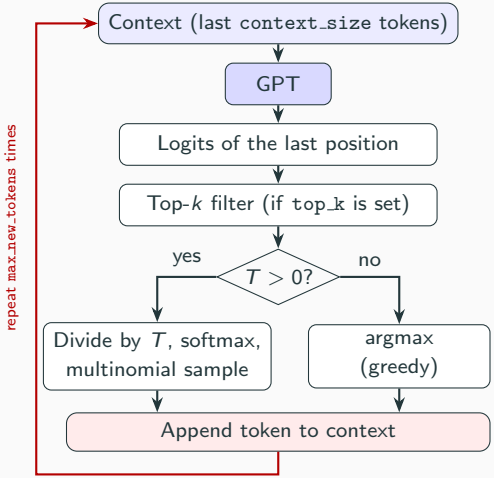
- **Masking with** $-\infty$ implements the indicator: $e^{-\infty} = 0$.
- It is a **renormalization** of the kept probabilities: a **conditional distribution** $P(X = v \mid X \in \mathcal{V}_k)$.
- For $T > 0$, z/T keeps the order of the logits $\Rightarrow$ $\mathcal{V}_k$ is the same before or after scaling.

Toy example ($k = 3,\ T = 1$):

$$\sum_{j\in\mathcal{V}_3} p_j = 0.061 + 0.572 + 0.358 = 0.991$$

$$\tilde{p}_{\text{forward}} = 0.572/0.991 = 0.578$$
$$\tilde{p}_{\text{toward}} = 0.358/0.991 = 0.361$$
$$\tilde{p}_{\text{closer}} = 0.061/0.991 = 0.061$$

Putting it together: the new `generate`



With `top_k`=25, $T = 1.4$:

```
Every effort moves you stand to work
on surprise, a one of us had gone
with ...
```

Very different from the **memorized** passage produced by greedy decoding.

Choosing the decoding parameters

Low temperature / small k

- Predictable, focused output.
- Factual Q&A, code, classification, extraction.

High temperature / large k

- Diverse, creative output (risk of incoherence).
- Brainstorming, stories, poetry.

Deterministic behavior
`temperature=0.0` (argmax) or `top_k=1` both make `generate` behave like `generate_text_simple`.

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

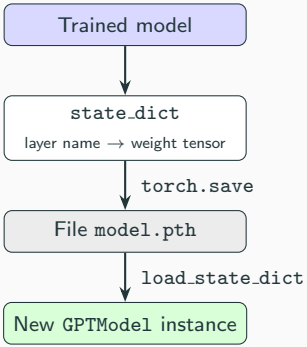
Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Saving and loading model weights



- Pretraining is expensive ⇒ save the model to avoid retraining.
- `state_dict`: dictionary mapping each layer to its parameters.
- For **inference**: load and call `model.eval()` (dropout off).
- To **continue training**: also save the **optimizer state**. AdamW keeps running moment estimates per weight; without them the optimizer resets and training may converge poorly.

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

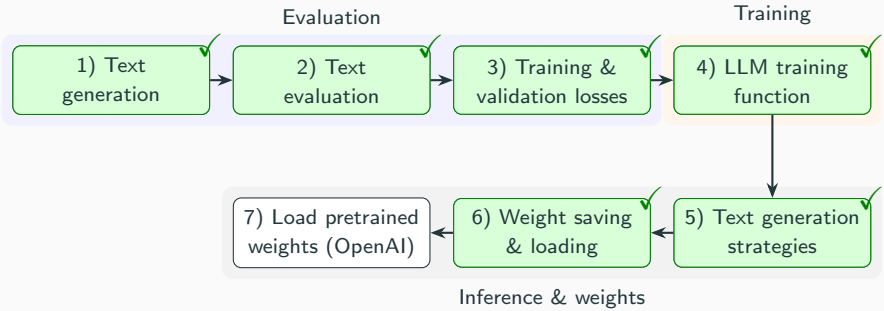
Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Progress so far



The GPT-2 checkpoints

- OpenAI released the GPT-2 weights $\Rightarrow$ no need to spend \$\$\$ on pretraining.
- Saved with **TensorFlow**; a helper downloads 7 files ($\approx$500 MB for 124M).
- Two dictionaries:
  - `settings`: architecture (`n_vocab`, `n_ctx`, `n_embd`, `n_head`, `n_layer`)
  - `params`: weight arrays (`wte`, `wpe`, `blocks`, `g`, `b`)
- e.g. `wte`: 50,257 $\times$ 768 token embeddings.

Model	Emb.	Layers	Heads
small (124M)	768	12	12
medium (355M)	1024	24	16
large (774M)	1280	36	20
XL (1558M)	1600	48	25

Same architecture; only the embedding size and the number of repetitions change.

Matching our configuration to GPT-2

Starting from `GPT_CONFIG_124M`, three changes are needed:

Setting	Ours	GPT-2	Why
Model size	—	from table	emb. dim, layers, heads
Context length	256	**1024**	OpenAI trained with 1024 tokens
Q, K, V bias	False	**True**	OpenAI used biases in the projections

- Modern LLMs drop the Q, K, V biases (no performance gain), but we must match the pretrained checkpoint.
- Each weight is copied with a **shape check**: a mismatch raises an error instead of failing silently.

Name mapping: OpenAI → GPTModel

OpenAI (params)	Our model	Note
wte	tok_emb.weight	token embeddings
wpe	pos_emb.weight	positional embeddings
blocks[b].attn.c_attn.{w,b}	att.W_query / W_key / W_value	split in 3, transpose
blocks[b].attn.c_proj.{w,b}	att.out_proj	transpose
blocks[b].mlp.c_fc.{w,b}	ff.layers[0]	768 → 3072
blocks[b].mlp.c_proj.{w,b}	ff.layers[2]	3072 → 768
blocks[b].ln_1 / ln_2.{g,b}	norm1 / norm2.{scale,shift}	γ, β
g, b	final_norm.{scale,shift}	
wte	out_head.weight	**weight tying**

OpenAI stores Q, K, V as **one fused matrix** (c_attn) ⇒ `np.split(..., 3, axis=-1)`. Its weights are stored as (in, out), so we **transpose** (`.T`) to match `nn.Linear`.

The linear algebra behind the conversion

1. Transposes. `nn.Linear` stores $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and computes

$$\mathbf{y} = \mathbf{x}W^\top + \mathbf{b}$$

OpenAI stores $W' \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and computes $\mathbf{y} = \mathbf{x}W' + \mathbf{b} \Rightarrow W = W'^\top$.

2. Fused Q, K, V. One matrix $W_{\text{attn}} \in \mathbb{R}^{768 \times 2304}$:

$$\begin{aligned}\mathbf{x}\,[\,W_Q \mid W_K \mid W_V\,] &= [\,\mathbf{x}W_Q \mid \mathbf{x}W_K \mid \mathbf{x}W_V\,] \\ &= [\,\mathbf{q} \mid \mathbf{k} \mid \mathbf{v}\,]\end{aligned}$$

Splitting the columns in 3 blocks of 768 recovers W_Q, W_K, W_V.

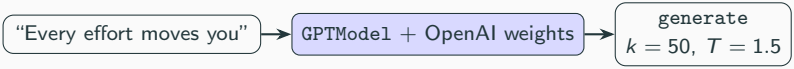
3. Weight tying. Token embedding matrix $E \in \mathbb{R}^{V \times d}$ (row v = embedding of token v). The output layer reuses it:

$$\mathbf{z}_t = E\,\mathbf{h}_t, \qquad z_{t,v} = \langle \mathbf{e}_v, \mathbf{h}_t \rangle$$

The logit of token v is the **dot-product similarity** between the hidden state and the embedding of v.

Parameter saving:
$V \cdot d = 50{,}257 \times 768 \approx 38.6\text{M}$ parameters.

Generating with the pretrained GPT-2



Output

```
Every effort moves you toward finding an ideal new way to practice
something!  What makes us want to be on top of that?
```

Coherent text $\Rightarrow$ the weights were loaded correctly (a single mistake in the mapping would break the model). This model is the starting point for **finetuning** in the next chapters.

Roadmap

Introduction

Evaluating generative text models

Training and validation losses

Training an LLM

Decoding strategies to control randomness

Saving and loading model weights

Loading pretrained weights from OpenAI

Summary and exercises

Summary

- LLMs generate text **one token at a time**; picking the most likely token is **greedy decoding**.
- **Cross entropy** (negative average log probability of the targets) measures text quality; **perplexity** $= e^{\mathcal{L}}$ is its interpretable version.
- **Training and validation losses** track progress and reveal **overfitting**.
- Pretraining = a standard deep learning loop: cross entropy + **AdamW**.
- **Temperature** and **top-k sampling** control the diversity and coherence of the generated text.
- Save `state_dicts` (model + optimizer) to reuse or continue training.
- Pretraining is costly, so we can **load OpenAI's GPT-2 weights** into our own `GPTModel`.

Exercises (1/2)

Exercise 5.1 – Sampling frequencies
Use `print_sampled_tokens` with temperatures 1, 0.1 and 5. How often is “pizza” sampled? Can you find a faster and more accurate way to compute it?

Exercise 5.2 – Decoding settings
Try different temperatures and top-k values. Which applications need low settings, and which need high ones?

Exercise 5.3 – Deterministic generation
Which settings of `generate` make it deterministic, like `generate_text_simple`?

Exercises (2/2)

Exercise 5.4 – Continue pretraining
Load the saved model and optimizer in a new session and train for one more epoch with `train_model_simple`.

Exercise 5.5 – Losses of the pretrained model
Compute the training and validation losses of the OpenAI GPT-2 model on *The Verdict*.

Exercise 5.6 – Bigger models
Load larger GPT-2 models (e.g. 1558M) and compare their generated text with the 124M model.

Practice: Pretraining a GPT model

Appendix: code

Appendix: code
- Evaluation
- Data and losses
- Training
- Decoding
- Saving and loading weights

Every main slide with a ▸ Code button links to its listing here; ◂ Back returns to that slide.

Code: model setup

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,
    "context_length": 256,  # was 1024
    "emb_dim": 768,
    "n_heads": 12,
    "n_layers": 12,
    "drop_rate": 0.1,
    "qkv_bias": False
}
torch.manual_seed(123)
model = GPTModel(GPT_CONFIG_124M)
model.eval()
```

- Same `GPTModel` as in Chapter 4.
- Only change: `context_length` $1024 \rightarrow 256$.
- Reduces compute and memory $\Rightarrow$ training fits on a **standard laptop**.
- At the end of the chapter we switch back to 1024 to load OpenAI's weights.

Back

Code: text ↔ token IDs

```python
import tiktoken

def text_to_token_ids(text, tokenizer):
    encoded = tokenizer.encode(text, allowed_special={'<|endoftext|>'})
    return torch.tensor(encoded).unsqueeze(0)       # add batch dimension

def token_ids_to_text(token_ids, tokenizer):
    flat = token_ids.squeeze(0)                      # remove batch dimension
    return tokenizer.decode(flat.tolist())

tokenizer = tiktoken.get_encoding("gpt2")
token_ids = generate_text_simple(model=model,
    idx=text_to_token_ids("Every effort moves you", tokenizer),
    max_new_tokens=10, context_size=GPT_CONFIG_124M["context_length"])
print(token_ids_to_text(token_ids, tokenizer))
```

Output (untrained model)

```
Every effort moves you rentingetic wasn refres RexMeCHicular stren
```

We need a **numerical measure** of "how good" the generated text is.

Code: inputs, targets and predictions

```
with torch.no_grad():
    logits = model(inputs)
probas = torch.softmax(logits, dim=-1)
print(probas.shape)
# torch.Size([2, 3, 50257])

token_ids = torch.argmax(probas, dim=-1,
                         keepdim=True)
```

- Shape `[batch, tokens, vocab]`.
- One probability distribution over **50,257 tokens** per position.
- Argmax gives the predicted token IDs.

Targets batch 1:	`effort moves you`
Outputs batch 1:	`Armed heNetflix`

Training goal: **maximize the probability** assigned to the correct target token at every position.

Back

Code: cross entropy

```
print(logits.shape)   # [2, 3, 50257]
print(targets.shape)  # [2, 3]

logits_flat  = logits.flatten(0, 1)  # [6,
    50257]
targets_flat = targets.flatten()     # [6]

loss = torch.nn.functional.cross_entropy(
           logits_flat, targets_flat)
print(loss)           # tensor(10.7722)
```

- `cross_entropy` performs steps 2–6 internally (softmax, selection, log, mean, negation).
- It takes **raw logits**, not probabilities.
- Flatten over the batch dimension: every token position is one "classification" example.
- Same value as the manual computation.

Back

Code: target probabilities

```
target_probas_1 = probas[0, [0, 1, 2], targets[0]]
target_probas_2 = probas[1, [0, 1, 2], targets[1]]
# Text 1: tensor([7.4541e-05, 3.1061e-05, 1.1563e-05])
# Text 2: tensor([3.9836e-05, 1.6783e-05, 4.7559e-06])
```

- With random weights, every probability is close to $1/50{,}257 \approx 0.00002$.
- We want these values to move **towards 1**.
- **How?** Update the weights via **backpropagation**, which needs a **loss function**: a single number that measures how far predictions are from the targets.

Back

Code: train / validation split

```
train_ratio = 0.90
split_idx = int(train_ratio * len(text_data))
train_data = text_data[:split_idx]
val_data   = text_data[split_idx:]

torch.manual_seed(123)
train_loader = create_dataloader_v1(
    train_data, batch_size=2,
    max_length=GPT_CONFIG_124M["context_length"],
    stride=GPT_CONFIG_124M["context_length"],
    drop_last=True, shuffle=True)

val_loader = create_dataloader_v1(
    val_data, batch_size=2,
    max_length=GPT_CONFIG_124M["context_length"],
    stride=GPT_CONFIG_124M["context_length"],
    drop_last=False, shuffle=False)
```

- Reuses `create_dataloader_v1` from Chapter 2.
- Result:
  - **9** training batches
  - **1** validation batch
- Each batch: `x`, `y` of shape `[2, 256]`.
- Only the **training** loader is shuffled.

Back

Code: loss of a batch and of a data loader

```
def calc_loss_batch(input_batch, target_batch, model, device):
    input_batch, target_batch = input_batch.to(device), target_batch.to(device)
    logits = model(input_batch)
    loss = torch.nn.functional.cross_entropy(logits.flatten(0, 1), target_batch.flatten())
    return loss

def calc_loss_loader(data_loader, model, device, num_batches=None):
    total_loss = 0.
    if num_batches is None:
        num_batches = len(data_loader)                      # all batches
    else:
        num_batches = min(num_batches, len(data_loader))    # only a subset (faster)
    for i, (input_batch, target_batch) in enumerate(data_loader):
        if i < num_batches:
            loss = calc_loss_batch(input_batch, target_batch, model, device)
            total_loss += loss.item()
        else:
            break
    return total_loss / num_batches                         # average loss
```

Before training: Training loss: **10.988** Validation loss: **10.981**

Back

Code: train_model_simple

```python
def train_model_simple(model, train_loader, val_loader, optimizer, device, num_epochs,
                       eval_freq, eval_iter, start_context):
    train_losses, val_losses, track_tokens_seen = [], [], []   # track progress
    tokens_seen, global_step = 0, -1

    for epoch in range(num_epochs):                            # 1) epochs
        model.train()
        for input_batch, target_batch in train_loader:         # 2) batches
            optimizer.zero_grad()                              # 3) reset gradients
            loss = calc_loss_batch(input_batch, target_batch, model, device)  # 4) loss
            loss.backward()                                    # 5) gradients
            optimizer.step()                                   # 6) update weights
            tokens_seen += input_batch.numel()
            global_step += 1
            if global_step % eval_freq == 0:                   # 7) optional evaluation
                train_loss, val_loss = evaluate_model(
                    model, train_loader, val_loader, device, eval_iter)
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                track_tokens_seen.append(tokens_seen)
                print(f"Ep {epoch+1} (Step {global_step:06d}): "
                      f"Train loss {train_loss:.3f}, Val loss {val_loss:.3f}")
        generate_and_print_sample(model, train_loader.dataset.tokenizer,   # 8) sample
                                  device, start_context)
    return train_losses, val_losses, track_tokens_seen
```

Code: monitoring helpers

```python
def evaluate_model(model, train_loader, val_loader, device, eval_iter):
    model.eval()                                    # dropout disabled
    with torch.no_grad():                           # no gradient tracking
        train_loss = calc_loss_loader(train_loader, model, device, num_batches=eval_iter)
        val_loss   = calc_loss_loader(val_loader,   model, device, num_batches=eval_iter)
    model.train()
    return train_loss, val_loss

def generate_and_print_sample(model, tokenizer, device, start_context):
    model.eval()
    context_size = model.pos_emb.weight.shape[0]
    encoded = text_to_token_ids(start_context, tokenizer).to(device)
    with torch.no_grad():
        token_ids = generate_text_simple(model=model, idx=encoded,
                                         max_new_tokens=50, context_size=context_size)
        decoded_text = token_ids_to_text(token_ids, tokenizer)
        print(decoded_text.replace("\n", " "))      # compact print format
    model.train()
```

`evaluate_model`: **numeric** estimate of progress. `generate_and_print_sample`: **qualitative** check of the text.

Back

Code: running the training

```
torch.manual_seed(123)
model = GPTModel(GPT_CONFIG_124M)
model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=0.0004, weight_decay=0.1)
num_epochs = 10
train_losses, val_losses, tokens_seen = train_model_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=num_epochs, eval_freq=5, eval_iter=1,
    start_context="Every effort moves you")
```

```
Ep 1 (Step 000000):  Train loss 9.781, Val loss 9.933
Every effort moves you,,,,,,,,,,,,.
Ep 2 (Step 000015):  Train loss 5.961, Val loss 6.616
Every effort moves you, and, and, and, and, and, and, and, ...
[...]
Ep 10 (Step 000085):  Train loss 0.391, Val loss 6.452
Every effort moves you know," was one of the axioms he laid down ...
```

About **5 minutes** on a laptop. The text goes from commas to grammatically correct sentences.

Back

Code: toy example and sampling

```
vocab = {"closer": 0, "every": 1,
         "effort": 2, "forward": 3,
         "inches": 4, "moves": 5,
         "pizza": 6, "toward": 7,
         "you": 8}
inverse_vocab = {v: k for k, v in vocab.items()}

# context: "every effort moves you"
next_token_logits = torch.tensor(
    [4.51, 0.89, -1.90, 6.75, 1.63,
     -1.62, -1.89, 6.28, 1.79])

probas = torch.softmax(next_token_logits, dim=0)
next_token_id = torch.argmax(probas).item()
print(inverse_vocab[next_token_id])  # forward
```

Replace `argmax` by **sampling** from the distribution:

```
torch.multinomial(probas, num_samples=1)
```

Sampling 1,000 times:

Token	Count
forward	582
toward	343
closer	73
inches	2
others	0

Back

Code: temperature scaling

```
def softmax_with_temperature(logits, temperature):
    scaled_logits = logits / temperature
    return torch.softmax(scaled_logits, dim=0)

temperatures = [1, 0.1, 5]   # original, sharper, flatter
scaled_probas = [softmax_with_temperature(next_token_logits, T)
                 for T in temperatures]
```

Back

Code: top-k

```
top_k = 3
top_logits, top_pos = torch.topk(next_token_logits, top_k)
# Top logits:    tensor([6.7500, 6.2800, 4.5100])
# Top positions: tensor([3, 7, 0])

new_logits = torch.where(
    condition=next_token_logits < top_logits[-1],  # below the k-th largest logit
    input=torch.tensor(float('-inf')),             # -> -inf
    other=next_token_logits)                       # otherwise keep the logit

topk_probas = torch.softmax(new_logits, dim=0)
# tensor([0.0615, 0., 0., 0.5775, 0., 0., 0., 0.3610, 0.])
```

Back

Code: `generate`

```python
def generate(model, idx, max_new_tokens, context_size,
             temperature=0.0, top_k=None):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]
        if top_k is not None:                    # top-k filtering
            top_logits, _ = torch.topk(logits, top_k)
            min_val = top_logits[:, -1]
            logits = torch.where(logits < min_val,
                torch.tensor(float('-inf')).to(logits.device),
                logits)
        if temperature > 0.0:                    # temperature + sampling
            logits = logits / temperature
            probs = torch.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
        else:                                    # greedy decoding
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

With `top_k`=25, `temperature`=1.4:

```
Every effort moves you stand to work on surprise, a one of us had gone with ...
```

Very different from the memorized passage produced by greedy decoding.

Note: `min_val` should be `top_logits[:, -1:]` (keep dim) so it broadcasts correctly for batch size > 1.

Back

Code: saving and loading the `state_dict`

```
# Save
torch.save(model.state_dict(), "model.
    pth")

# Load into a new instance
model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(torch.load("model.
    pth"))
model.eval()
```

- Pretraining is expensive $\Rightarrow$ save the model to avoid retraining.
- `state_dict`: a dictionary mapping each layer to its parameters.
- `.pth` is just a convention.
- `model.eval()` disables dropout for inference.

Back

Code: saving the optimizer too

```
torch.save({
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    }, "model_and_optimizer.pth")

checkpoint = torch.load("model_and_optimizer.pth")
model = GPTModel(GPT_CONFIG_124M)
model.load_state_dict(checkpoint["model_state_dict"])
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weight_decay=0.1)
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
model.train()
```

- AdamW keeps **extra state** per weight (running moment estimates).
- To **continue training** later, restore it; otherwise the optimizer resets and training may converge poorly.

Code: downloading GPT-2

- OpenAI released the GPT-2 weights $\Rightarrow$ no need to spend $$$ on pretraining.
- Weights were saved with **TensorFlow**: `pip install tensorflow>=2.15.0 tqdm>=4.66`

```
from gpt_download import download_and_load_gpt2   # helper from the book's repo
settings, params = download_and_load_gpt2(model_size="124M", models_dir="gpt2")

print(settings)
# {'n_vocab': 50257, 'n_ctx': 1024, 'n_embd': 768, 'n_head': 12, 'n_layer': 12}
print(params.keys())
# dict_keys(['blocks', 'b', 'g', 'wpe', 'wte'])
print(params["wte"].shape)    # (50257, 768) -> token embeddings
```

Downloads 7 files ($\approx$ 500 MB for 124M). Updated instructions:

`github.com/rasbt/LLMs-from-scratch`

Back

Code: matching the configuration

```
model_configs = {
  "gpt2-small (124M)":  {"emb_dim": 768,  "n_layers": 12, "
       n_heads": 12},
  "gpt2-medium (355M)": {"emb_dim": 1024, "n_layers": 24, "
       n_heads": 16},
  "gpt2-large (774M)":  {"emb_dim": 1280, "n_layers": 36, "
       n_heads": 20},
  "gpt2-xl (1558M)":    {"emb_dim": 1600, "n_layers": 48, "
       n_heads": 25},
}

model_name = "gpt2-small (124M)"
NEW_CONFIG = GPT_CONFIG_124M.copy()
NEW_CONFIG.update(model_configs[model_name])
NEW_CONFIG.update({"context_length": 1024})
NEW_CONFIG.update({"qkv_bias": True})

gpt = GPTModel(NEW_CONFIG)
gpt.eval()
```

- All GPT-2 sizes share the **same architecture**; only the embedding size and number of repetitions change.
- Context length back to **1024**.
- OpenAI used **bias** in the Q, K, V projections $\Rightarrow$ `qkv_bias=True`.
- Modern LLMs drop these biases: they don't help performance.

Back

Code: assign

```
def assign(left, right):
    if left.shape != right.shape:
        raise ValueError(f"Shape mismatch. Left: {left.shape}, Right: {right.
            shape}")
    return torch.nn.Parameter(torch.tensor(right))
```

- Checks that both tensors have the **same shape**.
- Returns the OpenAI array as a **trainable** PyTorch parameter.
- A mismatch raises an error immediately instead of failing silently.

Back

Code: load_weights_into_gpt (excerpt)

```python
def load_weights_into_gpt(gpt, params):
    gpt.pos_emb.weight = assign(gpt.pos_emb.weight, params['wpe'])
    gpt.tok_emb.weight = assign(gpt.tok_emb.weight, params['wte'])
    for b in range(len(params["blocks"])):
        blk, p = gpt.trf_blocks[b], params["blocks"][b]     # shorthand (slide only)
        q_w, k_w, v_w = np.split(p["attn"]["c_attn"]["w"], 3, axis=-1)
        blk.att.W_query.weight = assign(blk.att.W_query.weight, q_w.T)
        blk.att.W_key.weight   = assign(blk.att.W_key.weight,   k_w.T)
        blk.att.W_value.weight = assign(blk.att.W_value.weight, v_w.T)
        # ... same for the q/k/v biases (np.split, no transpose)
        blk.att.out_proj.weight = assign(blk.att.out_proj.weight, p["attn"]["c_proj"]["w"].T)
        blk.ff.layers[0].weight = assign(blk.ff.layers[0].weight, p["mlp"]["c_fc"]["w"].T)
        blk.ff.layers[2].weight = assign(blk.ff.layers[2].weight, p["mlp"]["c_proj"]["w"].T)
        # ... and the corresponding biases
        blk.norm1.scale = assign(blk.norm1.scale, p["ln_1"]["g"])
        blk.norm1.shift = assign(blk.norm1.shift, p["ln_1"]["b"])
        # ... norm2 <- ln_2
    gpt.final_norm.scale = assign(gpt.final_norm.scale, params["g"])
    gpt.final_norm.shift = assign(gpt.final_norm.shift, params["b"])
    gpt.out_head.weight  = assign(gpt.out_head.weight,  params["wte"])    # weight tying
```

Full code: Listing 5.5. `assign` catches shape errors; coherent output confirms the mapping.

Back

Code: generating with the pretrained GPT-2

```
load_weights_into_gpt(gpt, params)
gpt.to(device)

torch.manual_seed(123)
token_ids = generate(model=gpt,
    idx=text_to_token_ids("Every effort moves you", tokenizer),
    max_new_tokens=25, context_size=NEW_CONFIG["context_length"],
    top_k=50, temperature=1.5)
print(token_ids_to_text(token_ids, tokenizer))
```

Output

```
Every effort moves you toward finding an ideal new way to practice
something!  What makes us want to be on top of that?
```

Coherent text $\Rightarrow$ the weights were loaded correctly. This model is the starting point for **finetuning** in the next chapters.

Back